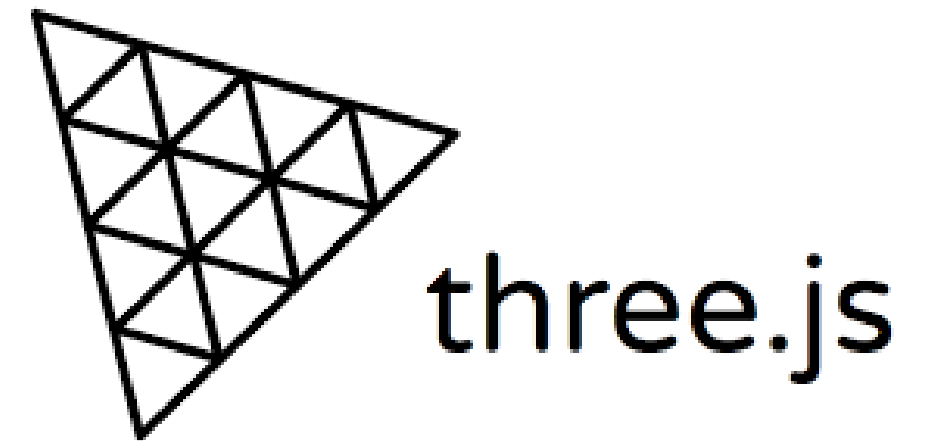
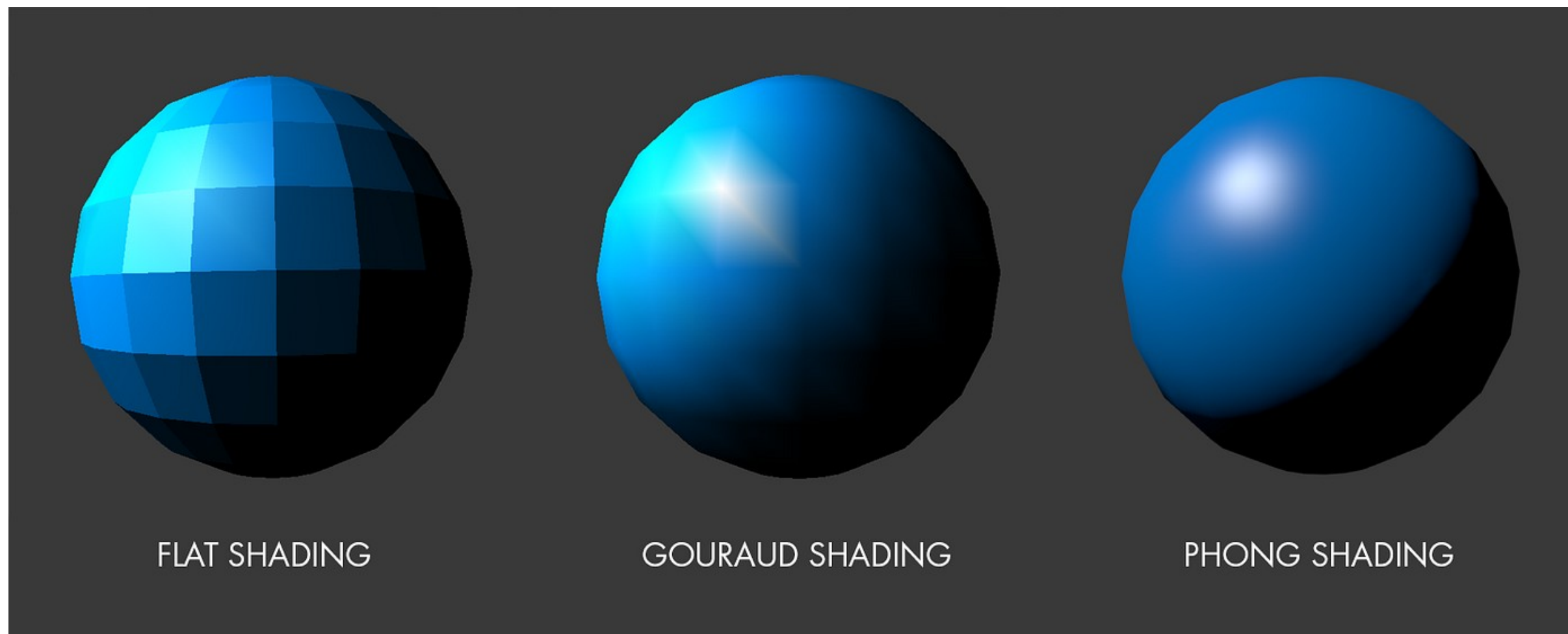


Introduction aux Shaders



Qu'est ce qu'un shader?

Un shader est un programme exécuté sur GPU qui permet de modifier le rendu d'un objet comme sa couleur ou sa géométrie. Ce programme est exécuté avant chaque rendu d'une scene et peut s'appliquer sur la scene entière ou sur un mesh spécifique



Qu'est ce qu'un shader?

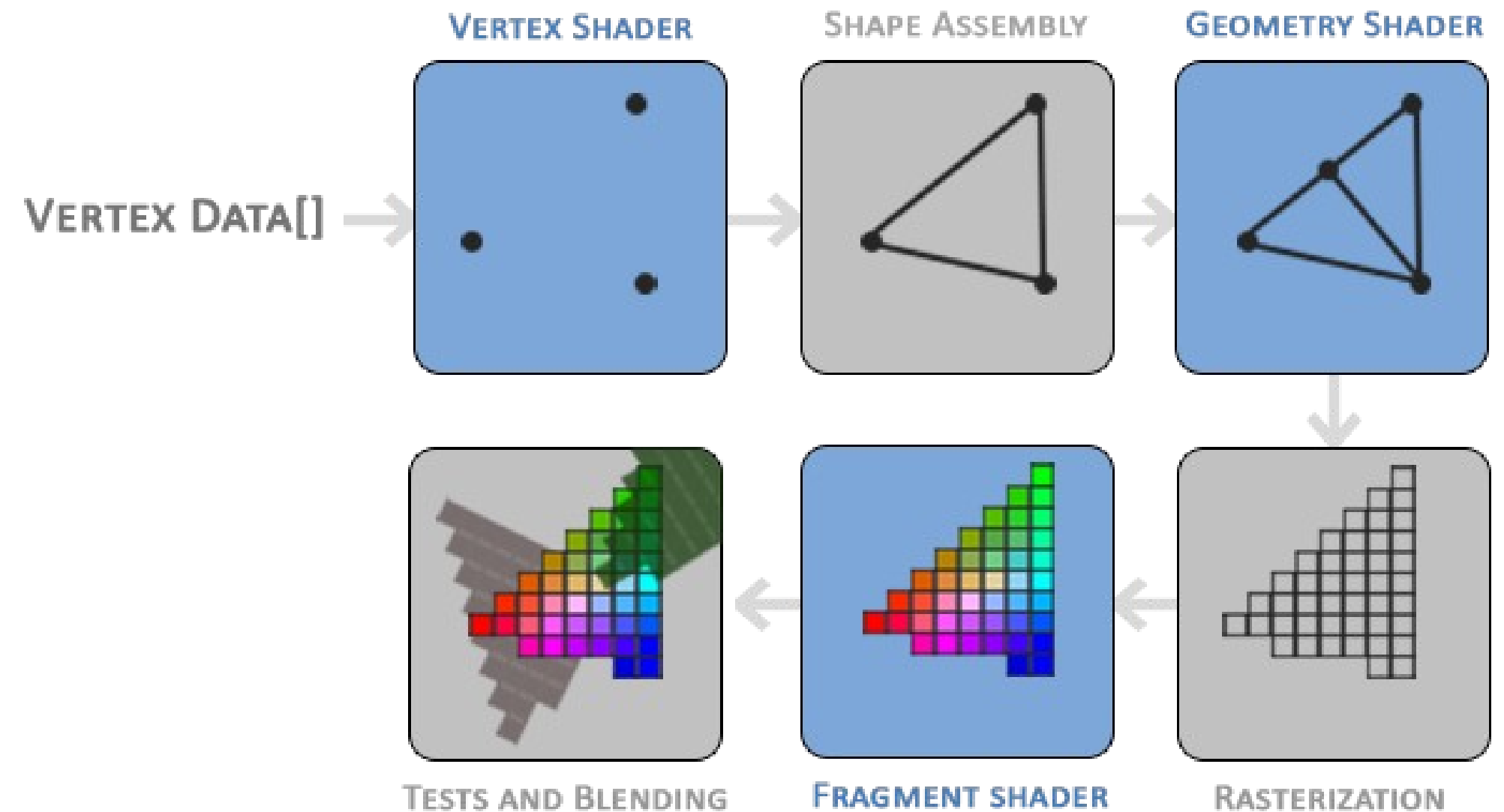
Un shader est composé de 2 programmes :

Le Vertex Shader :

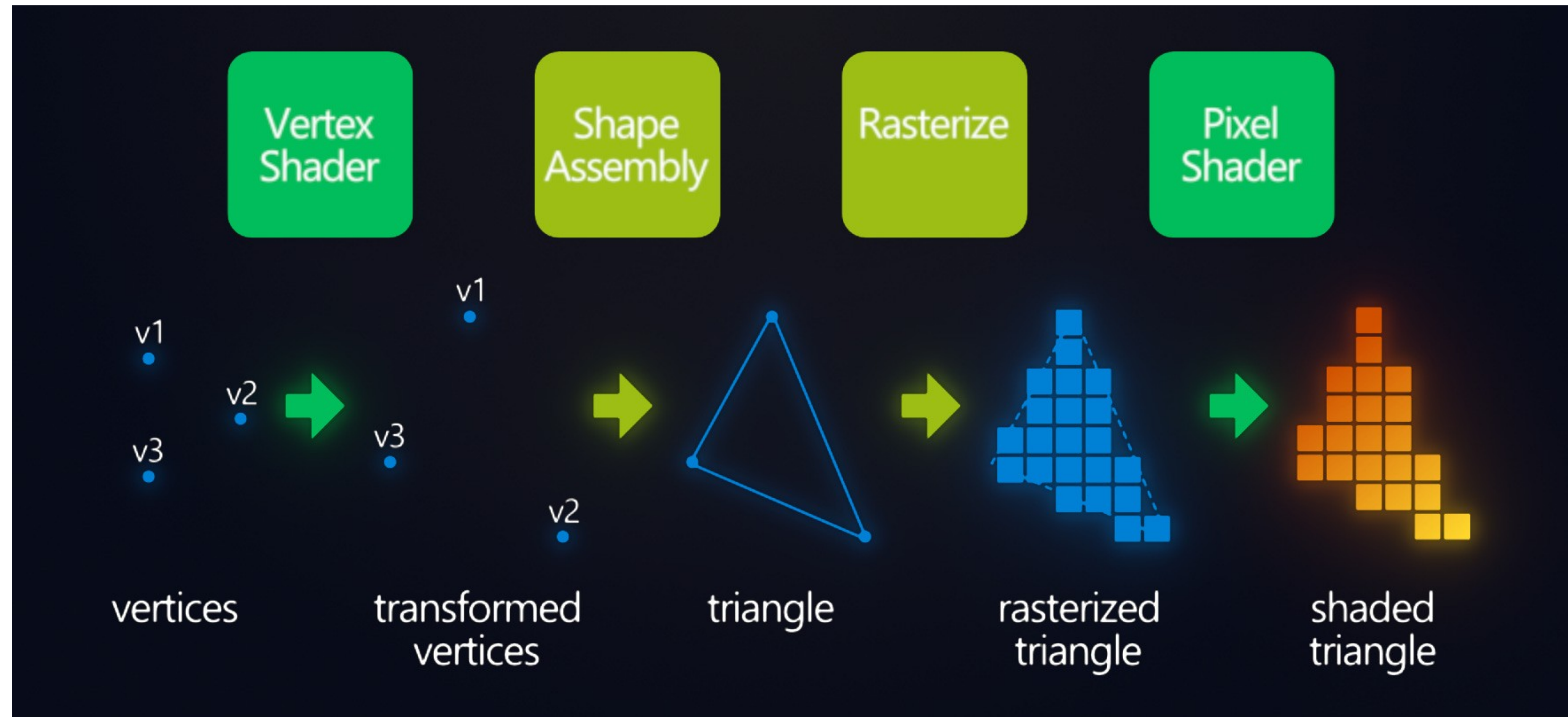
C'est un programme qui va s'exécuter sur chaque **sommet** d'un mesh et qui va permettre de modifier sa forme

Le Fragment Shader

C'est un programme qui va s'exécuter sur chaque **pixel** affiché par le mesh (Fragment) et qui va déterminer sa couleur



Fragment vs Vertex Shader



GLSL

Les shader sont codé en GLSL ("OpenGL Shading Language") , c'est une norme servant à écrire les programmes de shaders qui ressemble au language C qui tourne nativement sur les GPU

```
#ifdef GL_ES
precision mediump float;
#endif

uniform float u_time;

void main() {
    gl_FragColor = vec4(1.0,0.0,1.0,1.0);
}
```


GLSL

il est conçu pour le calcul graphique et possède des types natifs utiles en 3D :

- des vecteurs : `vec2`, `vec3`, `vec4`
- des matrices : `mat2`, `mat3`, `mat4`
- des fonctions mathématiques : `sin`, `cos`, `length`, `mix`, `clamp`

Shader minimal

Chaque shader est un programme en C qui contient une fonction main qui doit effectuer un calcul

Vertex Shader

```
void main() {  
    gl_Position = projectionMatrix  
        * modelViewMatrix * vec4(position, 1.0);  
}
```

Fragment Shader

```
void main() {  
    gl_FragColor = vec4(1.0, 0.0, 0.0, 1.0); // Red color  
}
```

ShaderMaterial

En ThreeJS, le ShaderMaterial permet d'appliquer un shader sur un mesh

```
let material = new THREE.ShaderMaterial({
  vertexShader: `
    varying vec2 vUv;

    void main() {
      vUv = uv;
      gl_Position = projectionMatrix * modelViewMatrix * vec4(position, 1.0);
    }
  `,
  fragmentShader: `
    varying vec2 vUv;

    void main() {
      gl_FragColor = vec4(vUv.x, vUv.y, 1.0, 1.0);
    }
  `,
});
```

```
const mesh = new THREE.Mesh(geometry, material);
scene.add(mesh);
```


Caractéristiques des shaders

- Les shaders s'exécutent sur chaque pixel de l'écran en **parallèles** sur le GPU sur des threads séparés
- Ils s'exécutent à chaque rendu d'une scene
- Ils sont **indépendant** les uns des autres
- Ils sont **amnésiques** : Il n'est pas possible de stocker de valeurs entre 2 exécutions



Vertex Shader

Le vertex shader est un programme qui prend en entrée la coordonnée d'un vertex sur la scene et qui doit calculer sa position finale

La position doit etre affecté sur la variable native **gl_Position**

gl_Position est un vecteur 3D composé de float (toujours écrire les valeurs avec un chiffre après la virgule). Il est le premier programme à s'exécuter

le vertex shader à accès aux variables natives de la géométrie comme position, normal, uv

```
vertexShader: `
    void main() {
        gl_Position = projectionMatrix * modelViewMatrix * vec4(position, 1.0);
    }
`
```

Fragment Shader

Le fragment shader est un programme qui prend en entrée la position du pixel sur l'écran et qui doit calculer sa couleur

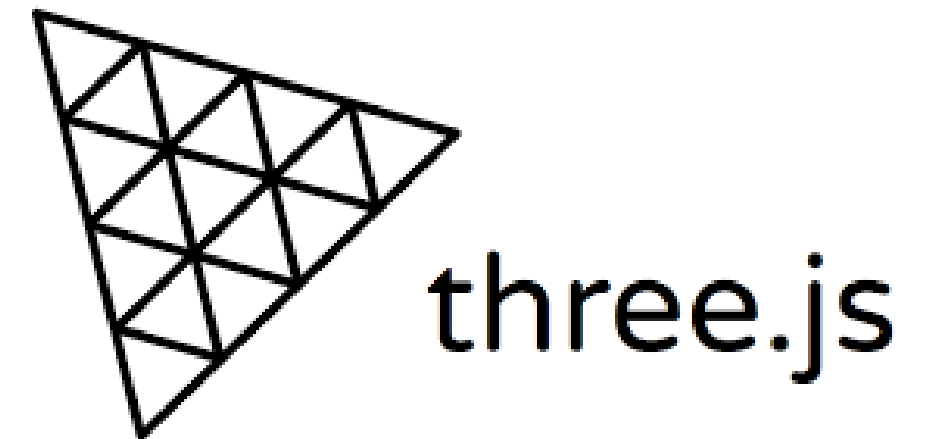
La couleur doit être affectée sur la variable native **gl_FragColor**

gl_FragColor est un vecteur 4D composé de float (toujours écrire les valeurs avec un chiffre après la virgule) au format R G B A

Le fragment shader a accès à la position du pixel sur l'écran `gl_FragCoord.xy` et sa couleur de base `gl_FragColor`. Il s'exécute en **dernier**

```
,  
fragmentShader: `  
    void main() {  
        gl_FragColor = vec4(1.0, 0.0, 0.0, 1.0);  
    }  
`,  
,
```

Shader et paramètres



Passages de paramètres : Uniform

Les shaders sont des programmes **statiques** qui ne peuvent pas stocker de valeurs
Cependant, ThreeJS a la possibilité de passer des paramètres aux shaders pour les rendre plus dynamique

Ces paramètres doivent être passés via un objet nommé uniform et peut contenir n'importe quelle variable JS

```
const material = new THREE.ShaderMaterial({  
  side: THREE.DoubleSide,  
  uniforms: {  
    time: { value: 0.0 }  
  },  
  vertexShader: `  
    ...  
  `,  
  fragmentShader: `  
    ...  
  `,  
});
```


Uniform et shader

Pour accéder à une propriété de l'uniform dans le fragment shader, il faut déclarer une variable avec le mot clé **uniform** qui porte le même nom et avec le bon type de donnée en variable globale du programme. Les uniform sont des variables **globales** pour tout le shader

```
const material = new THREE.ShaderMaterial({
  side: THREE.DoubleSide,
  uniforms: {
    time: { value: 0.0 }
  },
  vertexShader: `
    uniform float time;

    void main() {
      ...
    }
  `,
  fragmentShader: `
    uniform float time;
    void main() {
      ...
    }
  `,
});
```

Uniform et shader

Il est par exemple courant de passer le temps en uniform pour animer les shader

```
const uniforms = {  
  uTime: { value: 0.0 }  
};  
  
const material = new THREE.ShaderMaterial({  
  uniforms: uniforms,  
  vertexShader: `  
    void main() {  
      gl_Position = projectionMatrix * modelViewMatrix * vec4(position, 1.0);  
    }  
  `,  
  fragmentShader: `  
    uniform float uTime; // Declare uniform in GLSL  
    uniform vec3 uColor;  
    void main() {  
      // Use uTime for animation, e.g., pulsating color  
      vec3 finalColor = uColor * (0.5 + 0.5 * sin(uTime * 5.0));  
      gl_FragColor = vec4(finalColor, 1.0);  
    }  
  `,  
});
```

```
const clock = new THREE.Clock();  
  
function animate() {  
  material.uniforms.time.value = clock.getElapsedTime();  
  
  renderer.render(scene, camera);  
  requestAnimationFrame(animate);  
}
```


varying

Il est parfois nécessaire de passer des variables entre le vertex shader et le fragment shader
Pour cela il faut déclarer des variables de type **varying** dans les 2 programmes

```
let material = new THREE.ShaderMaterial({  
  vertexShader: `  
    varying vec2 vUv;  
  
    void main() {  
      vUv = uv;  
      gl_Position = projectionMatrix * modelViewMatrix * vec4(position, 1.0);  
    }  
  `,  
  fragmentShader: `  
    varying vec2 vUv;  
  
    void main() {  
      gl_FragColor = vec4(vUv.x, vUv.x, vUv.x, 1.0);  
    }  
  `,  
});
```


Attributes

Pour passer des variables spécifiques à chaque vertex, on peut utiliser des attributs déclarés sur la geometry

```
// Define unique colors for each vertex
const colors = new Float32Array([
  1.0, 0.0, 0.0, // Red for vertex 0
  0.0, 1.0, 0.0, // Green for vertex 1
  0.0, 0.0, 1.0, // Blue for vertex 2
]);
geometry.setAttribute('customColor', new THREE.BufferAttribute(colors, 3));
```

```
vertexShader: `
  attribute vec3 aColor;    // Custom attribute
  varying vec3 vColor;      // Passed to fragment shader

  void main() {
    vColor = aColor;
    gl_Position = projectionMatrix * modelViewMatrix * vec4(position, 1.0);
  }
`,
fragmentShader: `
  varying vec3 vColor; // Received from vertex shader

  void main() {
    gl_FragColor = vec4(vColor, 1.0);
  }
`
```

Attributes et Points

Les attributs permettent également de passer des valeurs spécifiques à des objets partageant la même geometry comme avec les instancedMesh et les Points

```
const color = new THREE.Color();

for ( let i = 0; i < particles; i ++ ) {

    color.setHSL( i / particles, 1.0, 0.5 );

    colors.push( color.r, color.g, color.b );

    sizes.push( 20 );

}

geometry.setAttribute( 'position', new THREE.Float32BufferAttribute( positions, 3 ) );
geometry.setAttribute( 'color', new THREE.Float32BufferAttribute( colors, 3 ) );
geometry.setAttribute( 'size', new THREE.Float32BufferAttribute( sizes, 1 ) );
```

```
let particles = new THREE.Points( geometry, shaderMaterial );
```

Attributes et InstancedMesh

```
const geometry = new THREE.InstancedBufferGeometry();
geometry.instanceCount = instances;

geometry.setAttribute( 'position', new THREE.Float32BufferAttribute( positions, 3 ) );
geometry.setAttribute( 'color', new THREE.InstancedBufferAttribute( new Float32Array( colors )
    , 4 ) );
```

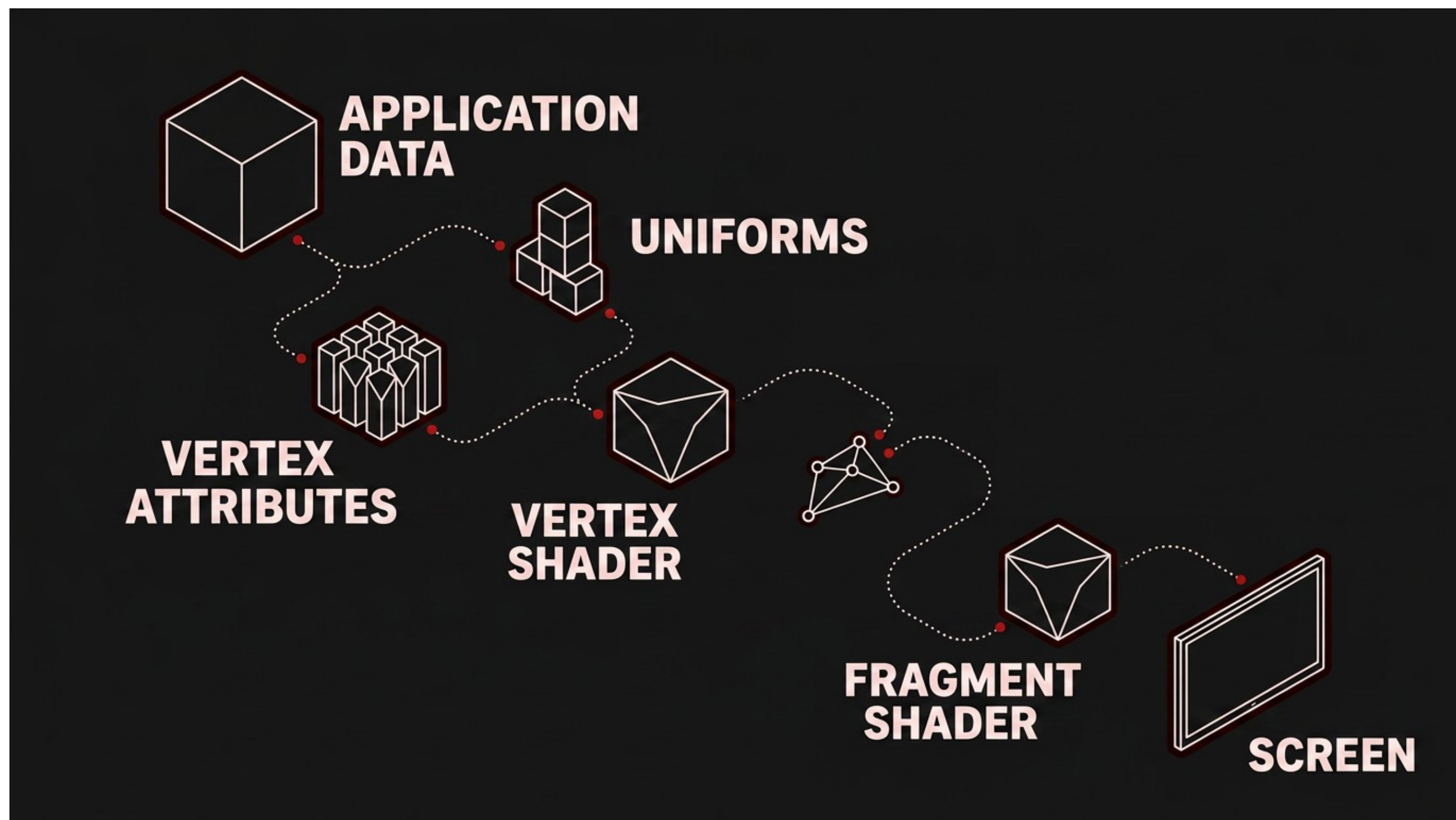
```
attribute vec3 offset;
attribute vec4 color;

varying vec3 vPosition;
varying vec4 vColor;

void main(){
    ...

    gl_Position = projectionMatrix * modelViewMatrix * vec4( vPosition, 1.0 );
}
```

Pipeline



UV Coordinates

Les coordonnées UV sont des coordonnées 2D associées aux vertex ou au fragment d'un shader.

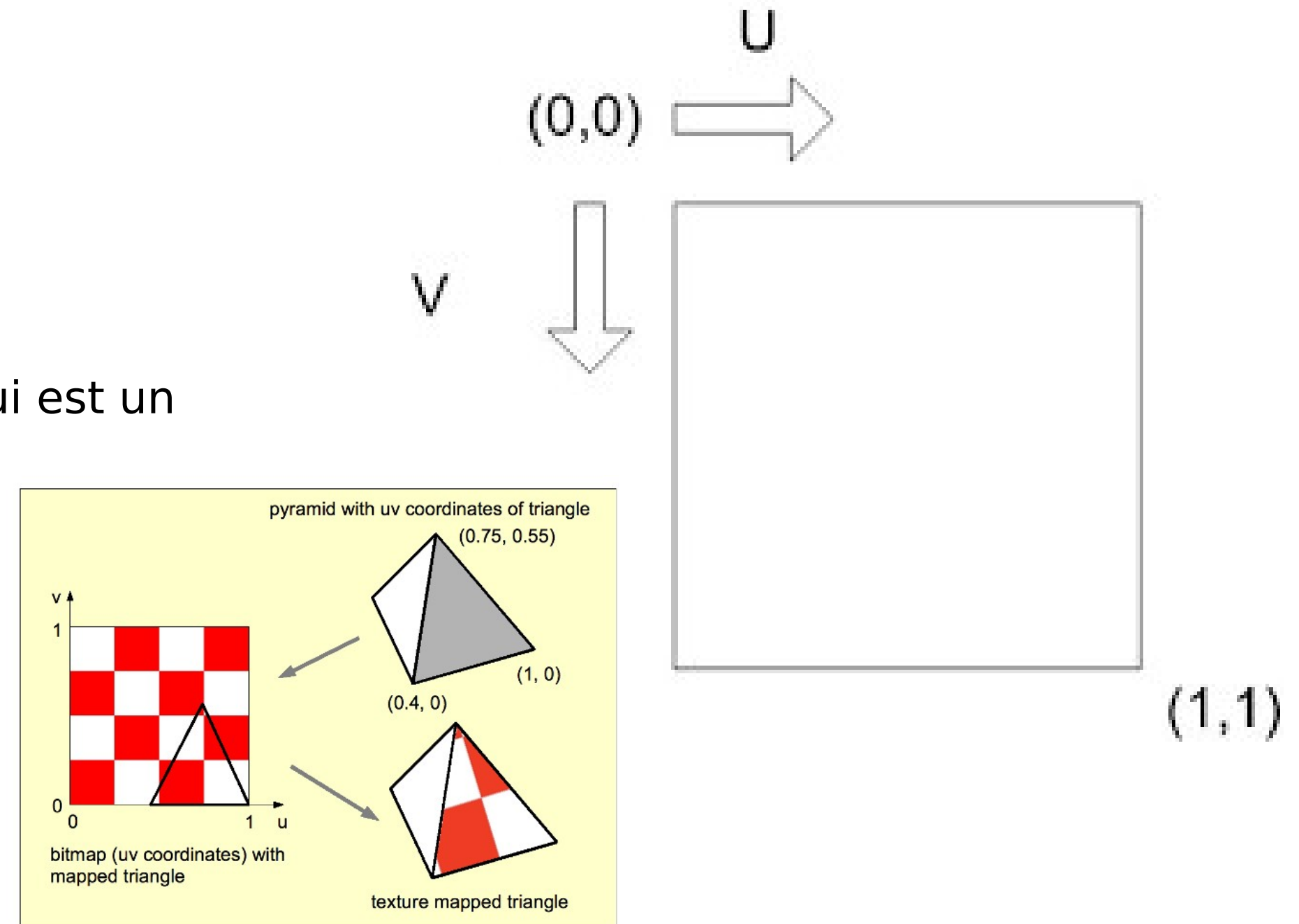
Elles servent à repérer une position sur la surface d'un objet, indépendamment de sa position dans la scène.

On les note généralement :

- U : axe horizontal
- V : axe vertical

Threejs passe la variable uv au vertex shader qui est un `vertex2d` qui va de 0,0 à 1,1

```
vec4 textureColor = texture2D(uTexture, vUv);
```



Shader et post processing

Il est possible de jouer un shader sur tout l'écran grace au **Shader Pass** qui peut etre ajouté à la chaine de post processing

```
const composer = new EffectComposer(renderer);
composer.addPass(new RenderPass(scene, camera));

const InvertShader = {
  uniforms: {
    'tDiffuse': { value: null } // The texture from the previous pass
  },
  vertexShader: `
    varying vec2 vUv;
    void main() {
      vUv = uv;
      gl_Position = projectionMatrix * modelViewMatrix * vec4(position, 1.0);
    }
  `,
  fragmentShader: `
    uniform sampler2D tDiffuse; // Input texture from the previous pass
    varying vec2 vUv;

    void main() {
      vec4 texel = texture2D(tDiffuse, vUv);
      // Invert the colors
      gl_FragColor = vec4(1.0 - texel.rgb, texel.a);
    }
  `
};
```

```
// Create and add the custom ShaderPass
const customPass = new ShaderPass(InvertShader);
composer.addPass(customPass);
```

Exercices